

Section 12: The Life of a Packet

Layer: L2/L3 | Reference: Odom Vol1 Ch1-3: TCP/IP Networking and Routing Fundamentals | *Original study notes* — CCNA 200-301

Tracing exactly what happens to a packet from source to destination - including what changes and what stays the same at each hop - is one of the most powerful mental models for the CCNA exam and for real troubleshooting.

Key Concepts

- Same subnet: source host ARPs directly for the destination host's MAC address; no router is involved
- Different subnet: source host ARPs for its default gateway's MAC address, since it can't reach the remote subnet directly
- At each router hop: the router strips the incoming Layer 2 frame header, examines the Layer 3 destination IP, consults its routing table, decrements TTL, and builds a brand-new Layer 2 header for the next hop
- Layer 3 addressing (source/destination IP) remains unchanged end-to-end across the entire path (barring NAT)
- Layer 2 addressing (source/destination MAC) changes at every single hop, since each hop is a new local-link transmission
- TTL (Time To Live) decrements by 1 at every router hop; if it reaches 0, the packet is discarded and an ICMP Time Exceeded message is sent back to the source - this is exactly how traceroute works
- ARP requests/replies never cross a router - they are local-segment only by design
- DNS resolves a hostname to an IP address (typically over UDP port 53) before any of the above even begins, if the application was given a name instead of an address
- Gratuitous ARP: a host announces its own IP-to-MAC binding unsolicited, often used to update switches/other hosts after a failover or address change
- Proxy ARP: a router answers an ARP request on behalf of a host on a different subnet, letting misconfigured hosts reach other subnets as if they were local (generally disabled by default on modern IOS)

Command Reference

Command	Purpose
<code>show arp</code>	View the local ARP cache (IP-to-MAC mappings)
<code>clear arp-cache</code>	Flush the ARP cache, forcing fresh ARP resolution on next traffic
<code>show ip route</code>	Verify what each router will decide for the next hop toward a destination
<code>tracert <dest></code>	Reveal each hop along the path by exploiting TTL expiration and ICMP Time Exceeded replies
<code>ip name-server <dns-ip></code>	Configure a router to use a DNS server for name resolution

Command	Purpose
<code>ip domain-lookup</code>	Enable DNS resolution on a router (enabled by default on most IOS versions)

Configuration Walkthrough

Configure DNS on a router and test resolution

```
R1(config)#ip domain-lookup
R1(config)#ip name-server 10.10.10.10
R1#ping R2
Translating "R2"...domain server (10.10.10.10) [OK]
```

Observe ARP cache being rebuilt after a clear

```
R1#clear arp-cache
R1#show arp
! Cache will be empty until traffic triggers new ARP requests
```

Worked Scenario

Q: PC-A (10.1.1.10/24) pings PC-B (10.1.2.10/24) through Router R1. Describe exactly what addressing changes happen along the way.

PC-A recognizes 10.1.2.10 is on a different subnet, so it ARPs for its default gateway (R1's LAN interface) and sends the frame with source MAC=PC-A, destination MAC=R1, source IP=PC-A, destination IP=PC-B. R1 receives the frame, strips the L2 header, checks its routing table for 10.1.2.0/24, decrements TTL, and builds a NEW L2 frame with source MAC=R1, destination MAC=PC-B (resolved via R1's own ARP on that segment), while the source/destination IP addresses remain unchanged throughout the entire path.

Common Mistakes / Gotchas

- Believing IP addresses change at each hop the way MAC addresses do - they don't (without NAT)
- Forgetting ARP requests/replies are confined to the local segment and never cross a router
- Assuming traceroute relies on something exotic rather than simply exploiting TTL expiration and ICMP Time Exceeded messages

Exam Tips

★ *The 'L3 stays the same end-to-end, L2 changes every hop' rule is one of the single most tested conceptual facts on the exam*

★ *Know that ARP is strictly local-segment, which is why hosts ARP for the gateway, not the remote host, when crossing subnets*